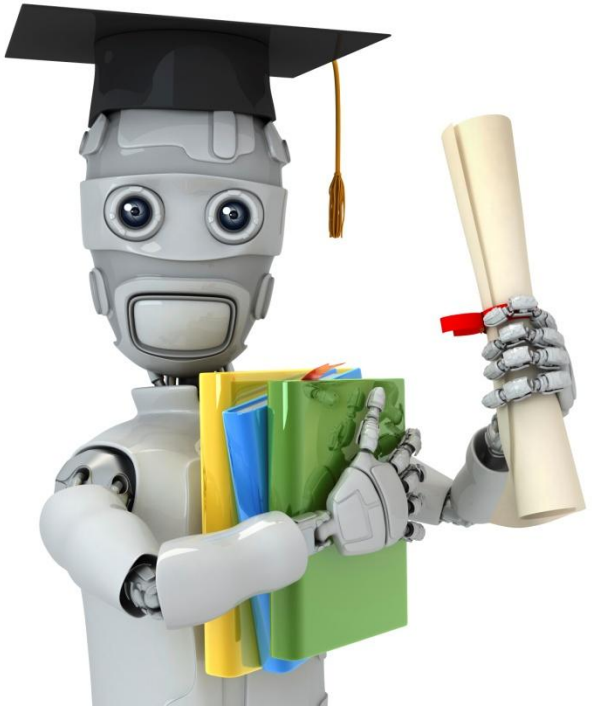
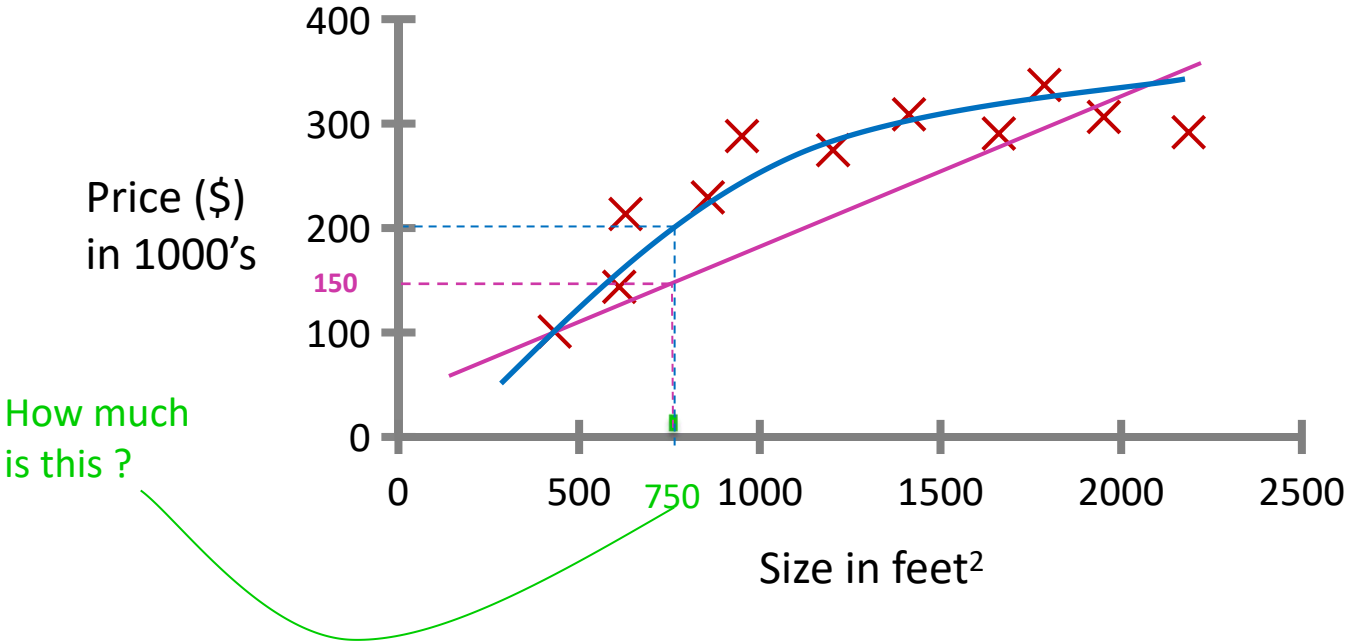


Machine Learning

Supervised and Unsupervised Learning

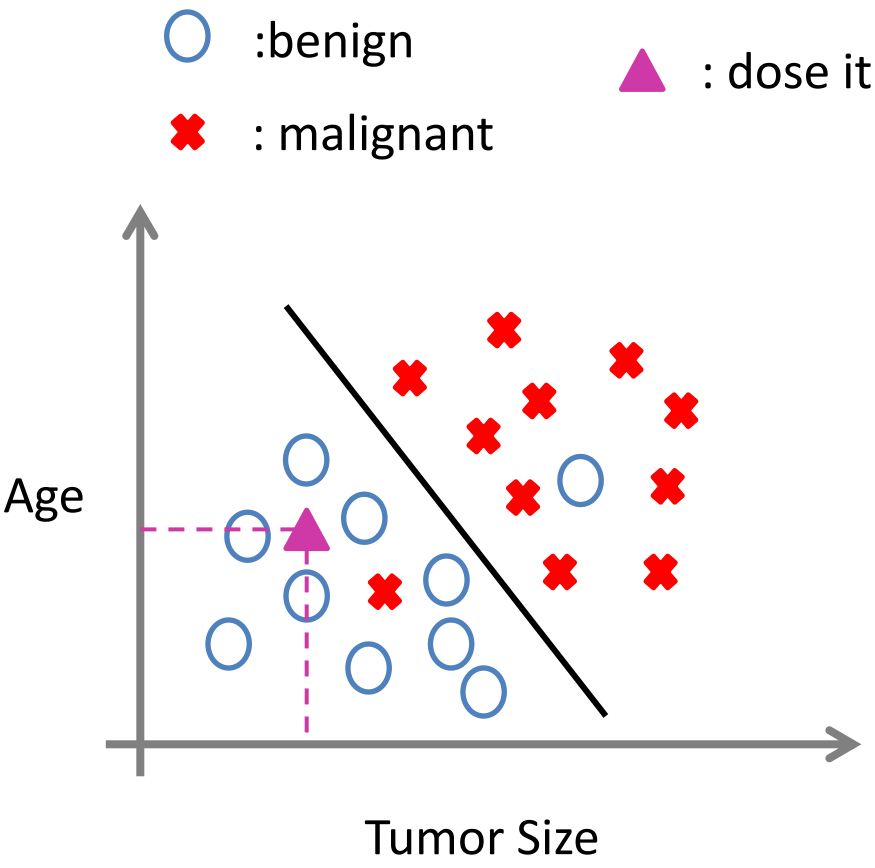
Housing price prediction.



Supervised Learning

"right answers" given

Regression: Predict continuous valued output (price)



Use two features, age and tumor size.

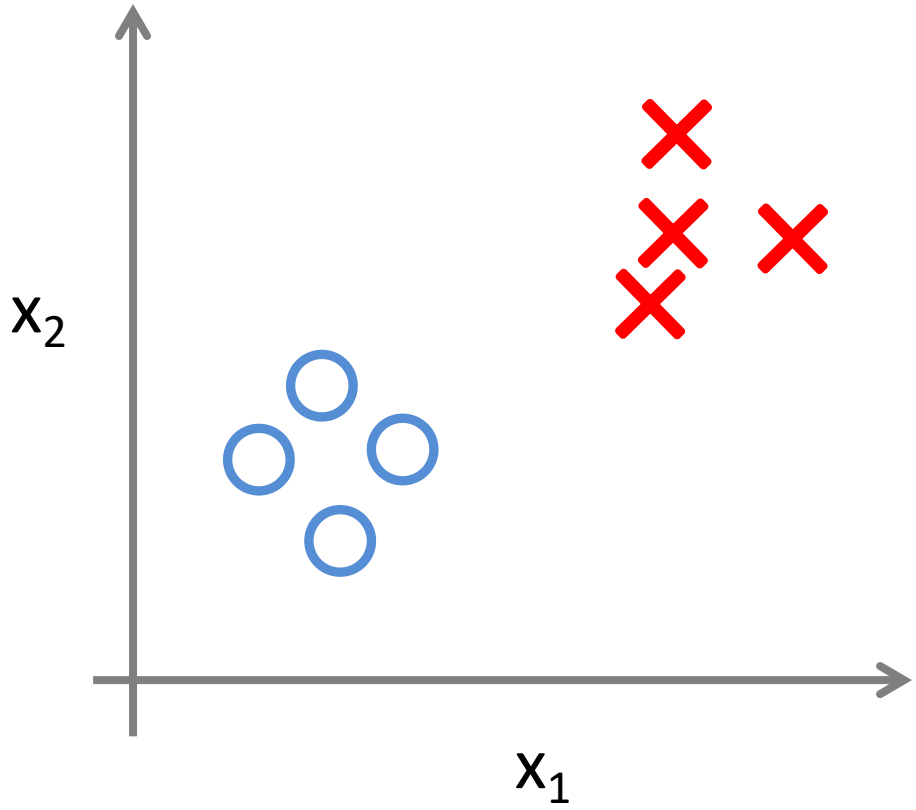
Supervised learning

--right answers given

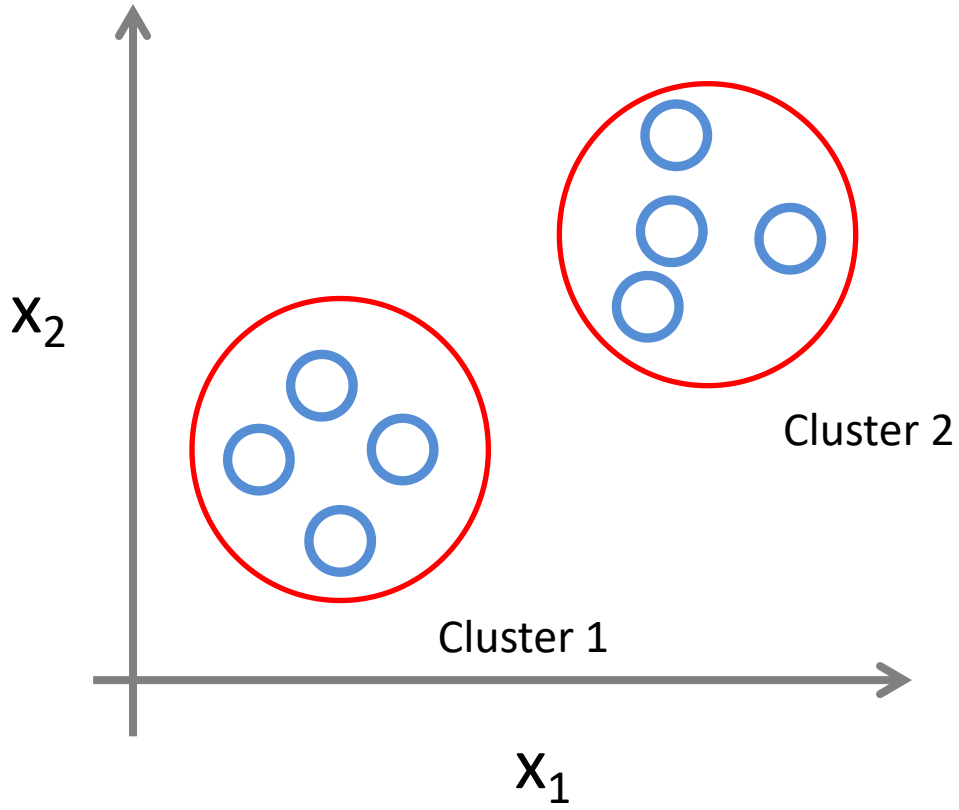
Classification

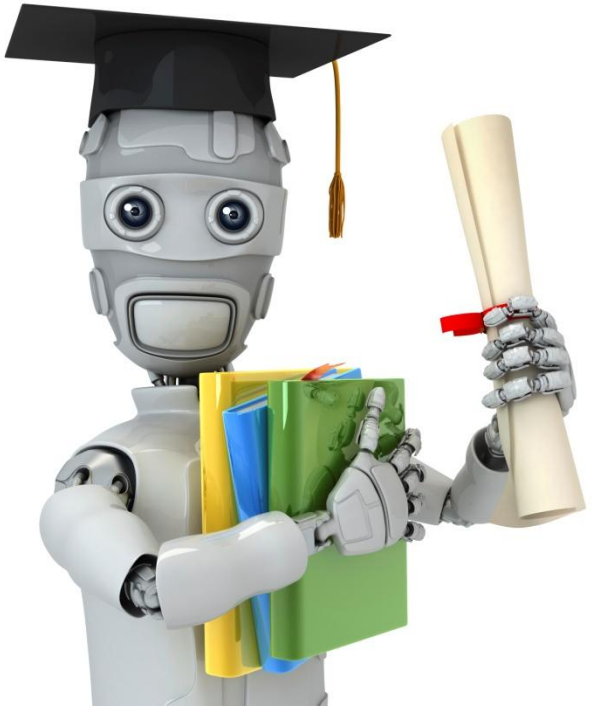
--predict discrete values

Supervised Learning



Unsupervised Learning





Machine Learning

Linear Regression

Multiple features (variables).

Size (feet^2)	Number of bedrooms	Number of floors	Age of home (years)	Price ($1000)
2104	5	1	45	460
1416	3	2	40	232
1534	3	2	30	315
852	2	1	36	178
…	…	…	…	…

Multiple features (variables).

Size (feet²) x_1	Number of bedrooms x_2	Number of floors x_3	Age of home (years) x_4	Price ($1000) y
2104	5	1	45	460
1416	3	2	40	232
1534	3	2	30	315
852	2	1	36	178
…	…	…	…	…

M=47

n=4

Notation:

n = number of features

$x^{(i)}$ = input (features) of ith training example.

$x_j^{(i)}$ = value of feature j in i^{th} training example.

$$x^{(2)} = \begin{bmatrix} 1416 \\ 3 \\ 2 \\ 40 \end{bmatrix}$$

$$x_3^{(2)} = 2$$

Hypothesis:

$$h_\theta(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_3 + \theta_4 x_4$$

E.g. $h_\theta(x) = 80 + 0.1x_1 + 0.02x_2 + 3x_3 - 2x_4$

$$h_\theta(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_n x_n$$

For convenience of notation, define $x_0 = 1.$ → $x_0^{(i)} = 1$

$$X = \begin{bmatrix} x_0 \\ x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} \in R^{n+1} \qquad \theta = \begin{bmatrix} \theta_0 \\ \theta_1 \\ \theta_2 \\ \vdots \\ \theta_n \end{bmatrix} \in R^{n+1}$$

$$h_\theta(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_n x_n$$

$$= \boldsymbol{\theta^T X}$$

$[\theta_0, \theta_1, \ldots, \theta_n]$ → $\boldsymbol{\theta^T}$

$\begin{bmatrix} x_0 \\ x_1 \\ \vdots \\ x_n \end{bmatrix}$ → $\boldsymbol{X}$

Multivariate linear regression.

Hypothesis: $h_\theta(x) = \theta^T x = \theta_0 x_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_n x_n$

Parameters: $\theta_0, \theta_1, \ldots, \theta_n$

Cost function:

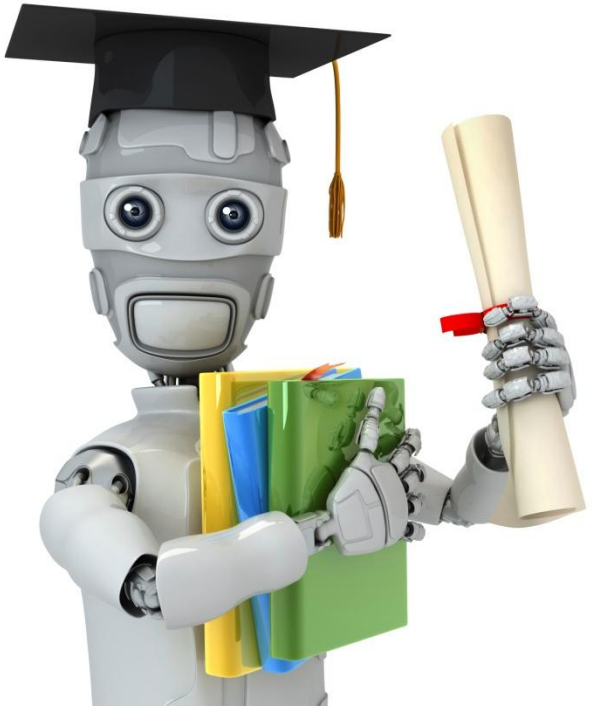
$$J(\theta_0, \theta_1, \ldots, \theta_n) = \frac{1}{2m} \sum_{i=1}^{m} (h_\theta(x^{(i)}) - y^{(i)})^2$$

Gradient descent:

Repeat $\{$

$$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta_0, \ldots, \theta_n)$$

$\}$ (simultaneously update for every $j = 0, \ldots, n$)



Machine Learning

Logistic Regression

Classification

Classification

Email: Spam / Not Spam?
Online Transactions: Fraudulent (Yes / No)?
Tumor: Malignant / Benign ?

$y \in \{0, 1\}$

0: “Negative Class” (e.g., benign tumor)
1: “Positive Class” (e.g., malignant tumor)

Classification: y = 0 or 1

Logistic Regression: $0 \leq h_\theta(x) \leq 1$

(a classification algorithm)

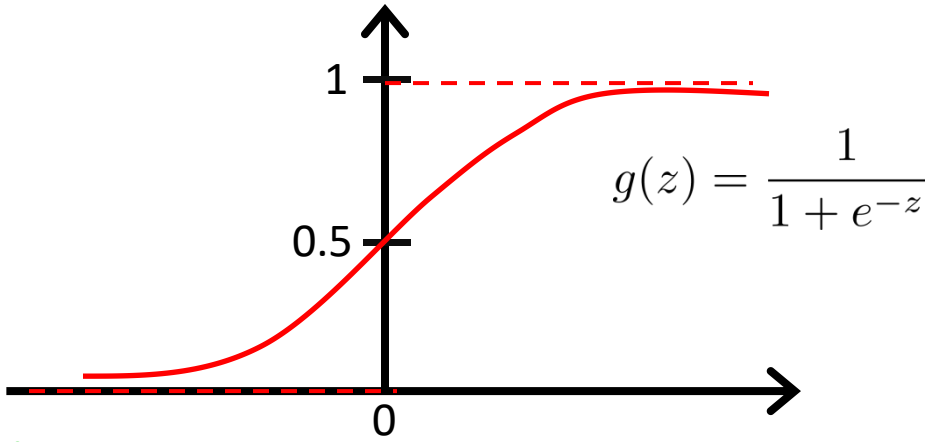
Logistic Regression Model

Want $0 \leq h_\theta(x) \leq 1$

$$h_\theta(x) = g(\theta^T x)$$

$$g(z) = \frac{1}{1+e^{-z}}$$

$$h_\theta(x) = \frac{1}{1+e^{-\theta^T x}}$$



Sigmoid function
Logistic function

Mean the same thing

Interpretation of Hypothesis Output

$h_\theta(x)$ = estimated probability that y = 1 on input x

Example: If $x = \begin{bmatrix} x_0 \\ x_1 \end{bmatrix} = \begin{bmatrix} 1 \\ \text{tumorSize} \end{bmatrix}$

$$h_\theta(x) = 0.7$$

Tell patient that 70% chance of tumor being malignant

$h_\theta(x) = P(y = 1|x; \theta)$

"probability that y = 1, given x, parameterized by θ"

$$P(y = 0|x; \theta) + P(y = 1|x; \theta) = 1$$
$$P(y = 0|x; \theta) = 1 - P(y = 1|x; \theta)$$

Training set: $\{(x^{(1)}, y^{(1)}), (x^{(2)}, y^{(2)}), \cdots, (x^{(m)}, y^{(m)})\}$

m examples $\quad x \in \begin{bmatrix} x_0 \\ x_1 \\ \cdots \\ x_n \end{bmatrix} \quad x_0 = 1, y \in \{0, 1\}$

$$h_\theta(x) = \frac{1}{1 + e^{-\theta^T x}}$$

How to choose parameters θ ?

Logistic regression cost function

$$J(\theta) = \frac{1}{m}\sum_{i=1}^{m} \text{Cost}(h_\theta(x^{(i)}), y^{(i)})$$

$$= -\frac{1}{m}[\sum_{i=1}^{m} y^{(i)} \log h_\theta(x^{(i)}) + (1 - y^{(i)}) \log (1 - h_\theta(x^{(i)}))]$$

To fit parameters θ:

$$\min_\theta J(\theta)$$

To make a prediction given new x:

Output $h_\theta(x) = \frac{1}{1+e^{-\theta^T x}}$

$p(y = 1|x; \theta)$

Gradient Descent

$$J(\theta) = -\frac{1}{m}[\sum_{i=1}^{m} y^{(i)} \log h_\theta(x^{(i)}) + (1 - y^{(i)}) \log (1 - h_\theta(x^{(i)}))]$$

Want $\min_\theta J(\theta)$:

Repeat $\{$

$$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta)$$

$\}$ (simultaneously update all θ_j)

Gradient Descent

$$J(\theta) = -\frac{1}{m}[\sum_{i=1}^{m} y^{(i)} \log h_\theta(x^{(i)}) + (1 - y^{(i)}) \log (1 - h_\theta(x^{(i)}))]$$

Want $\min_\theta J(\theta)$:

Repeat $\{$

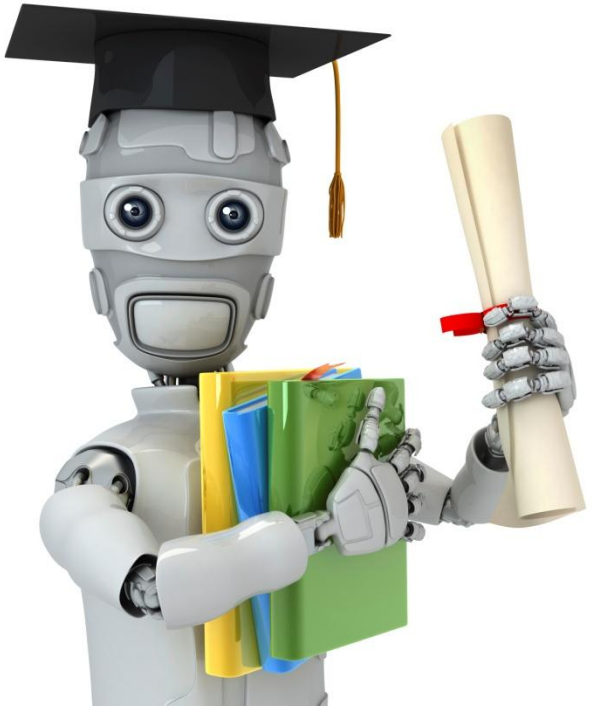
$$\theta_j := \theta_j - \alpha \sum_{i=1}^{m} (h_\theta(x^{(i)}) - y^{(i)}) x_j^{(i)}$$

(simultaneously update all θ_j)

$\}$

$$h_\theta(x) = \frac{1}{1 + e^{-\theta^T x}}$$

Algorithm looks identical to linear regression!



Machine Learning

Logistic Regression

Multi-class classification: One-vs-all

Multiclass classification

Email foldering/tagging: Work, Friends, Family, Hobby

y=1 y=2 y=3 y=4

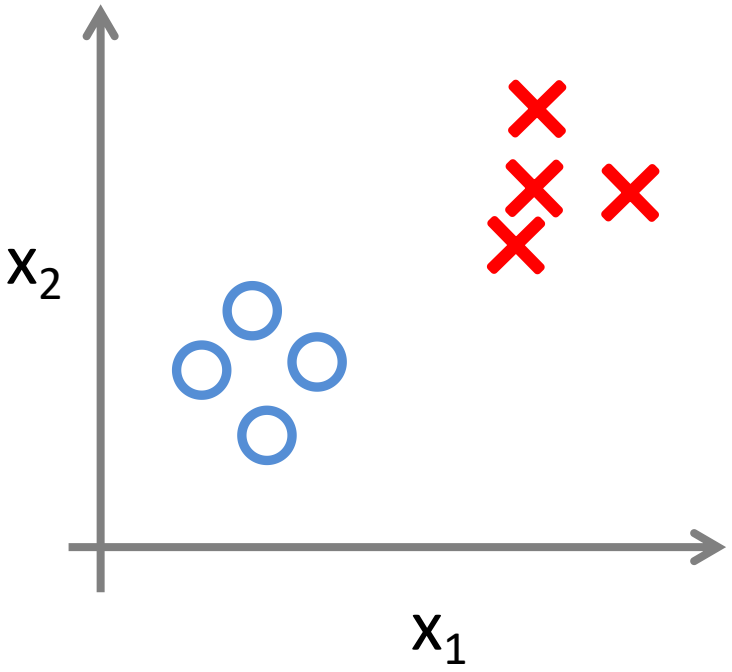
Medical diagrams: Not ill, Cold, Flu

y=1 y=2 y=3

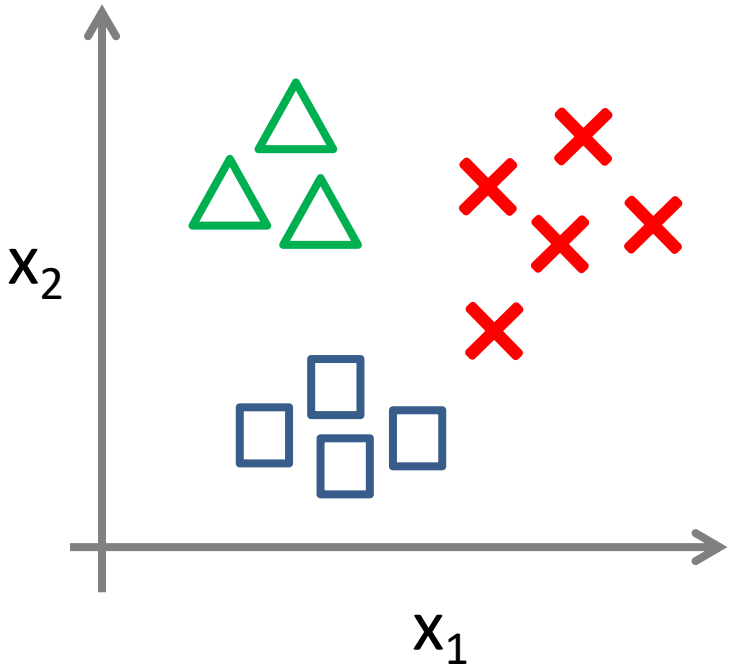
Weather: Sunny, Cloudy, Rain, Snow

y=1 y=2 y=3 y=4

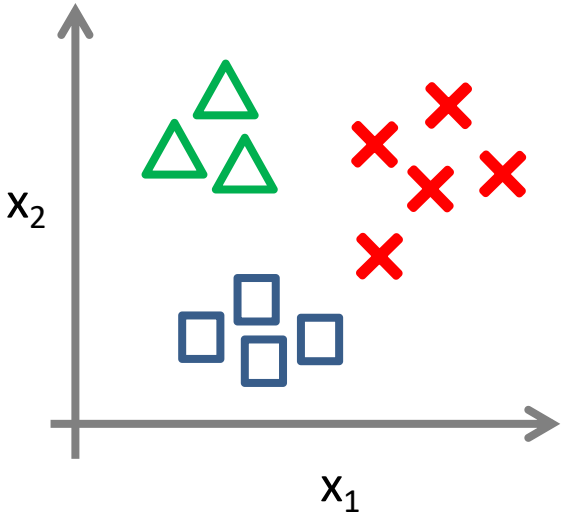
Binary classification:



Multi-class classification:



One-vs-all (one-vs-rest):

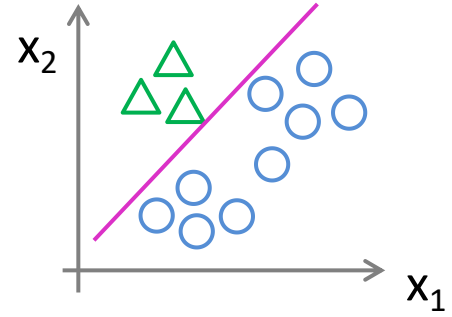


Class 1: △

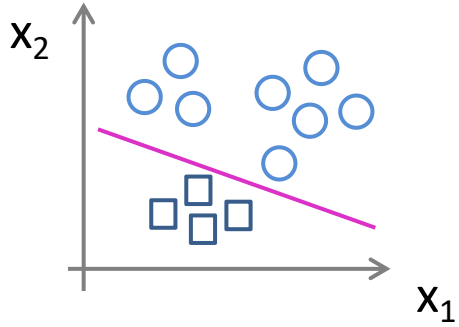
Class 2: □

Class 3: ×

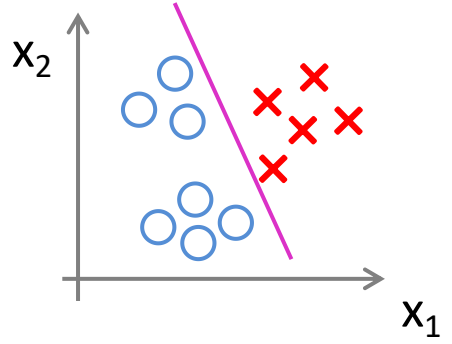
$h_\theta^{(i)}(x) = P(y = i|x;\theta) \qquad (i = 1, 2, 3)$



$h_\theta^{(1)}(x)$



$h_\theta^{(2)}(x)$



$h_\theta^{(3)}(x)$

One-vs-all

Train a logistic regression classifier $h_\theta^{(i)}(x)$ for each class i to predict the probability that $y = i$.

On a new input x, to make a prediction, pick the class i that maximizes

$$\max_i h_\theta^{(i)}(x)$$

Outline

- Probabilistic Perspective on Regression
- Probabilistic Perspective on Classification

Perspective from Conditional Probability

- The goal of regression and classification is to predict the *possible output* y given the input data $\boldsymbol{x}$

$$\boldsymbol{x} \xrightarrow{\text{predict}} y$$

- In the previous regression and classification, the prediction is given by some deterministic functions

Regression: $f(\boldsymbol{x}) = \boldsymbol{x}\boldsymbol{w}$

Classification: $f(\boldsymbol{x}) = \sigma(\boldsymbol{x}\boldsymbol{w})$

From the perspective of probability, to predict the output y given $\boldsymbol{x}$, we just need to model *the conditional probability*

$$p(y|\boldsymbol{x})$$

- With the conditional probability $p(y|\boldsymbol{x})$, the output can be predicted as

Mean: $\hat{y} = \int y p(y|\boldsymbol{x}) dy$

or

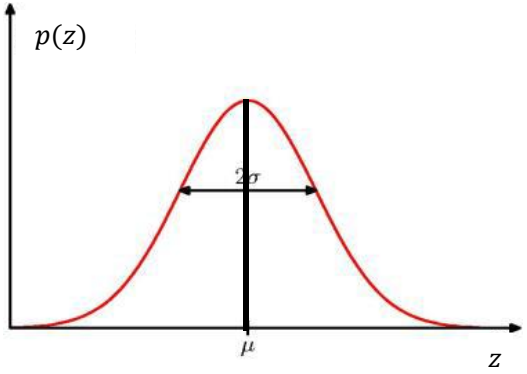
MAP: $\hat{y} = \arg\max_{y} p(y|\boldsymbol{x})$

Gaussian Distribution

- Univariate Gaussian distribution

$$p(z) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left[-\frac{1}{2}\frac{(z-\mu)^2}{\sigma^2}\right] \triangleq \mathcal{N}(z; \mu, \sigma^2)$$

  - μ is the mean
  - $\sigma^2 = E[(z-\mu)^2]$ *is the variance*
  - σ is the standard deviation



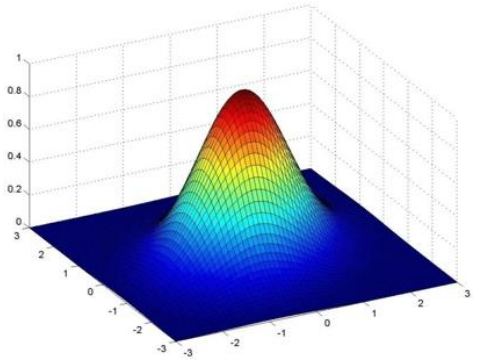
Bell shape

- μ is the *peak* and *central* of the distribution
- σ determine the spread of the distribution

- Multivariate Gaussian distribution

$$p(\boldsymbol{z}) = \frac{1}{(2\pi)^{D/2}|\boldsymbol{\Sigma}|^{1/2}} \exp\left\{-\frac{1}{2}(\boldsymbol{z}-\boldsymbol{\mu})^T\boldsymbol{\Sigma}^{-1}(\boldsymbol{z}-\boldsymbol{\mu})\right\} \triangleq \mathcal{N}(\boldsymbol{z};\boldsymbol{\mu},\boldsymbol{\Sigma})$$

  - D is the dimension
  - $\boldsymbol{\mu} \in \mathbb{R}^D$ is the mean vector
  - $\boldsymbol{\Sigma} \in \mathbb{R}^{D\times D}$ is the covariance matrix, and $|\boldsymbol{\Sigma}|$ is its determinant



$$\boldsymbol{\Sigma} = E[(\boldsymbol{z}-\boldsymbol{\mu})(\boldsymbol{z}-\boldsymbol{\mu})^T]$$

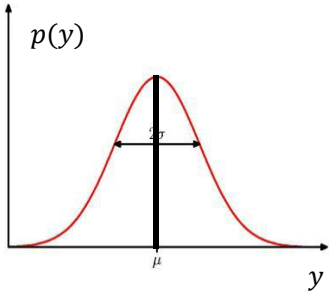
- $\boldsymbol{\mu}$ controls the peak or the central point
- $\boldsymbol{\Sigma}$ controls the shapes of the distribution

Linear Regression

- From the probabilistic perspective , to make prediction, we only need to specify the conditional probability distribution $p(y|\boldsymbol{x})$. For regression, we assume the distribution is a normal distribution

$$p(y|\boldsymbol{x};\boldsymbol{w}) = \frac{1}{\sqrt{2\pi\sigma^2}}\exp\left[-\frac{1}{2}\frac{(y-\boldsymbol{x}\boldsymbol{w})^2}{\sigma^2}\right]$$

$$= \mathcal{N}(y;\boldsymbol{x}\boldsymbol{w},\ \sigma^2)$$



- We make prediction by using the mean of the distribution, *i.e.*,

$$\hat{y} = \boldsymbol{x}\boldsymbol{w}$$

Is the $\boldsymbol{w}$ obtained here the same as that in traditional regression?

- Training the model aims to find the parameter $\mathbf{w}$ that maximizes the log-probability, that is,

$$\max_{\mathbf{w}} \log p(y|\boldsymbol{x};\boldsymbol{w})$$

Log-likelihood function

- From the expression of $p(y|\boldsymbol{x};\boldsymbol{w})$, we obtain

$$\log p(y|\boldsymbol{x};\boldsymbol{w}) = -\frac{1}{2}\frac{(y-\boldsymbol{x}\boldsymbol{w})^2}{\sigma^2} + constant$$

Thus, maximizing the log-likelihood $\log p(y|\boldsymbol{x};\boldsymbol{w})$ is equivalent to

$$\min_{\boldsymbol{w}} (y-\boldsymbol{x}\boldsymbol{w})^2,$$

which is the same as the loss used in the regression

- For N training samples $\left(\boldsymbol{x}^{(i)}, y^{(i)}\right)$, by assuming they are *i.i.d.*, we can obtain their joint conditional pdf

$$p\left(y^{(1)}, \cdots, y^{(N)} \middle| \boldsymbol{x}^{(1)}, \cdots, \boldsymbol{x}^{(N)}\right) = \prod_{i=1}^{N} \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left[-\frac{1}{2}\frac{\left(y^{(i)} - \boldsymbol{x}^{(i)}\boldsymbol{w}\right)^2}{\sigma^2}\right]$$

- The log-likelihood function is

$$\log p\left(y^{(1)}, \cdots, y^{(N)} \middle| \boldsymbol{x}^{(1)}, \cdots, \boldsymbol{x}^{(N)}\right) = -\frac{1}{2\sigma^2}\sum_{i=1}^{N}\left(y^{(i)} - \boldsymbol{x}^{(i)}\boldsymbol{w}\right)^2 + constant$$

- Maximizing the log-likelihood $\log p\left(y^{(1)}, \cdots, y^{(N)} \middle| \boldsymbol{x}^{(1)}, \cdots, \boldsymbol{x}^{(N)}\right)$ is equivalent to minimize

$$L(\boldsymbol{w}) = \sum_{i=1}^{N}\left(y^{(i)} - \boldsymbol{x}^{(i)}\boldsymbol{w}\right)^2,$$

which is the same as the loss used in the regression

- From the perspective of probability, linear regression is actually equivalent to
  - Modeling: assuming *conditional distribution to be Gaussian*
  - Training: training the model by *maximizing the log-likelihood*

Bernoulli Distribution

- The Bernoulli distribution

$$p(z) = \begin{cases} \mu, & \text{if } z = 1 \\ 1 - \mu, & \text{if } z = 0 \end{cases}$$

where $\mu \in [0, 1]$ is the probability of being 1

- The $p(z)$ can be concisely expressed as

$$p(z) = \mu^z \cdot (1 - \mu)^{1-z}$$

where $z = 0$ or 1

Binary Classification

- To achieve binary classification, the conditional probability is assumed to be a Bernoulli distribution

$$p(y|\boldsymbol{x}) = \big(\sigma(\boldsymbol{xw})\big)^{y} \cdot \big(1 - \sigma(\boldsymbol{xw})\big)^{1-y}$$

where $\mu = \sigma(\boldsymbol{xw})$; and $y = 0$ or 1

- The training objective is to maximize the log-likelihood function

$$\log p(y|\boldsymbol{x}) = y \log \sigma(\boldsymbol{xw}) + (1-y) \log\big(1 - \sigma(\boldsymbol{xw})\big)$$

Recall that the logistic regression minimizes

$$cross\ entropy \triangleq -\, y \log \sigma(\boldsymbol{xw}) - (1-y) \log\big(1 - \sigma(\boldsymbol{xw})\big)$$

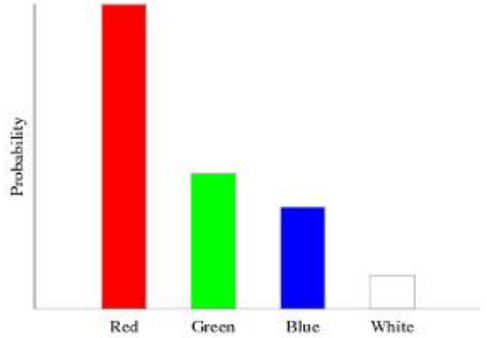
Maximizing $\log p(y|\boldsymbol{x})$ is equivalent to minimize the cross entropy

- The logistic regression is equivalent to
  - Modeling: assuming *Bernoulli conditional distribution* for the output
  - Training: training the model by *maximizing the log-likelihood*

Categorical Distribution

- The categorical distribution

$$p(\boldsymbol{z} = onehot_k) = \mu_k$$



  - where $onehot_i = [0, \cdots, 0, 1, 0, \cdots, 0]$ is the a vector with the i-th element being the only nonzero element 1
  - $\sum_{k=1}^{K} \mu_k = 1$

- The distribution can be equivalently written as

$$p(\boldsymbol{z}) = \prod_{k=1}^{K} \mu_k^{z_k}$$

where $\boldsymbol{z}$ is a one-hot vector

Multiclass Classification

- Modeling: By setting the probability

$$\mu_k = softmax_k(\boldsymbol{xW}),$$

the conditional probability distribution is assumed to be the categorical distribution

$$p(\boldsymbol{y}|\boldsymbol{x}) = \prod_{k=1}^{K} [softmax_k(\boldsymbol{xW})]^{y_k}$$

- Training: Given a training sample $(\boldsymbol{x}, \boldsymbol{y})$, the model is trained by maximizing the log-likelihood function

$$\log p(\boldsymbol{y}|\boldsymbol{x}) = \sum_{k=1}^{K} y_k \cdot \log\left(softmax_k(\boldsymbol{xW})\right)$$

$$= - \ cross\ entropy$$

Summary

- The regression, logistic and multi-class logistic regressions are equivalent to

1) assume different conditional pdfs for the outputs y

   - Regression: *Gaussian distribution*
   - Logistic regression: *Bernoulli distribution*
   - Multiclass logistic regression: *Categorical distribution*

2) maximize the log-likelihood functions

Example: Image Classification

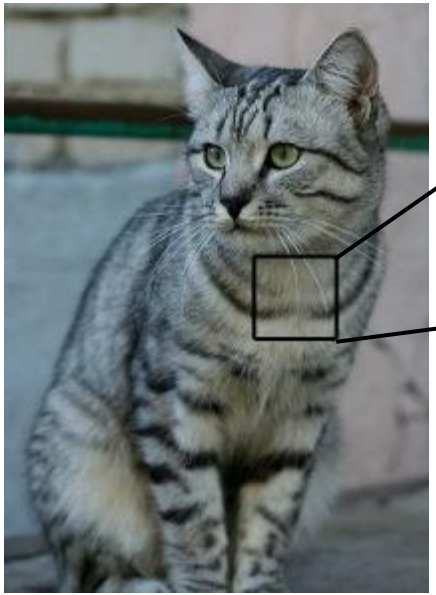


This image by Nikita is licensed under CC-BY 2.0

(assume given a set of possible labels)
{dog, cat, truck, plane, ...}

► cat

The Problem: Semantic Gap



This image by Nikita is licensed under CC-BY 2.0

```
[[105 112 108 111 104  99 106  99  96 103 112 119 104  97  93  87]
 [ 91  98 102 106 104  79  98 103  99 105 123 136 110 105  94  85]
 [ 76  85  98 105 128 105  87  96  95  99 115 112 106 103  99  85]
 [ 99  81  81  93 120 131 127 100  95  98 102  99  96  93 101  94]
 [106  91  61  64  69  91  88  85 101 107 109  98  75  84  96  95]
 [114 108  85  55  55  69  64  54  64  87 112 129  98  74  84  91]
 [133 137 147 103  65  81  80  65  52  54  74  84 102  93  85  82]
 [128 137 144 140 109  95  86  70  62  65  63  63  60  73  86 101]
 [125 133 148 137 119 121 117  94  65  79  80  65  54  64  72  98]
 [127 125 131 147 133 127 126 131 111  96  89  75  61  64  72  84]
 [115 114 109 123 150 148 131 118 113 109 100  92  74  65  72  78]
 [ 89  93  98  97 108 147 131 118 113 114 113 109 106  95  77  80]
 [ 63  77  86  81  77  79 102 123 117 115 117 125 125 130 115  87]
 [ 62  65  82  89  78  71  80 101 124 126 119 101 107 114 131 119]
 [ 63  65  75  88  89  71  62  81 120 138 135 105  81  98 110 118]
 [ 87  65  71  87 106  95  69  45  76 130 126 107  92  94 105 112]
 [118  97  82  86 117 123 116  66  41  51  95  93  89  95 102 107]
 [164 146 112  80  82 120 124 104  76  48  45  66  88 101 102 109]
 [157 170 157 120  93  86 114 132 112  97  69  55  70  82  99  94]
 [130 120 134 161 139 100 109 110 121 134 114  87  65  53  69  86]
 [120 112  96 117 150 144 120 115 104 107 102  93  87  81  72  79]
 [123 107  96  86  83 112 153 149 122 109 104  75  80 107 112  99]
 [122 121 102  80  82  86  94 117 145 148 153 102  58  78  92 107]
 [122 164 148 103  71  56  78  83  93 103 119 139 102  61  69  84]]
```

What the computer sees

An image is a tensor of integers between [0, 255]:

e.g. 800 x 600 x 3
(3 channels RGB)

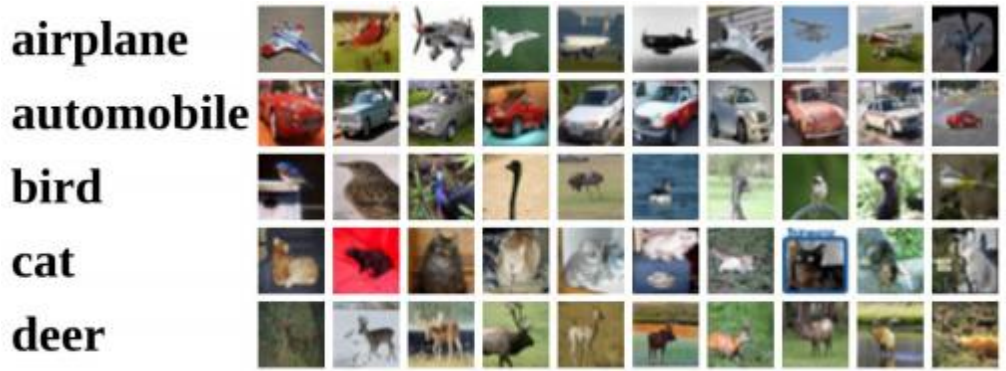
Machine Learning: Data-Driven Approach

1. Collect a dataset of images and labels
2. Use Machine Learning algorithms to train a classifier
3. Evaluate the classifier on new images

```
def train(images, labels):
  # Machine learning!
  return model
```

```
def predict(model, test_images):
  # Use model to predict labels
  return test_labels
```

Example training set



Multiclass Linear Classifier

Parametric Approach

Image



Array of **32x32x3** numbers
(3072 numbers total)

f(**x**,**W**)

W
parameters
or weights

10 numbers giving
class scores

Parametric Approach: Linear Classifier

$$f(x,W) = Wx$$

Image



Array of **32x32x3** numbers
(3072 numbers total)

f(**x**,**W**)

10 numbers giving class scores

W

parameters or weights

Parametric Approach: Linear Classifier

$$f(x, W) = Wx$$

3072x1

10x1

10x3072

Image



Array of **32x32x3** numbers
(3072 numbers total)

f(**x**,**W**)

10 numbers giving class scores

W

parameters or weights

Parametric Approach: Linear Classifier

$$f(x,W) = Wx + b$$

3072x1

10x1

10x3072

10x1

Image



Array of **32x32x3** numbers
(3072 numbers total)

f(**x**,**W**)

10 numbers giving class scores

W

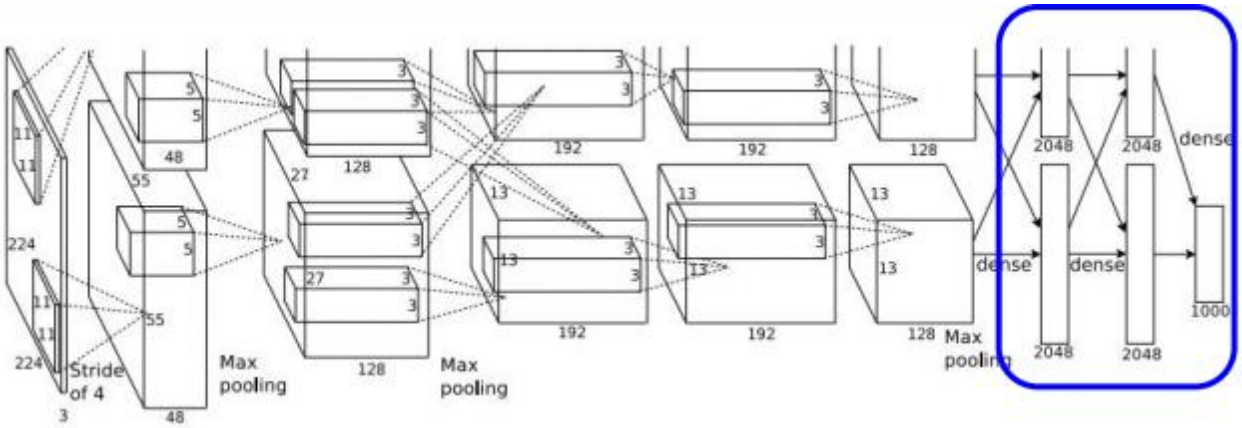
parameters or weights

Neural Network

Linear classifiers

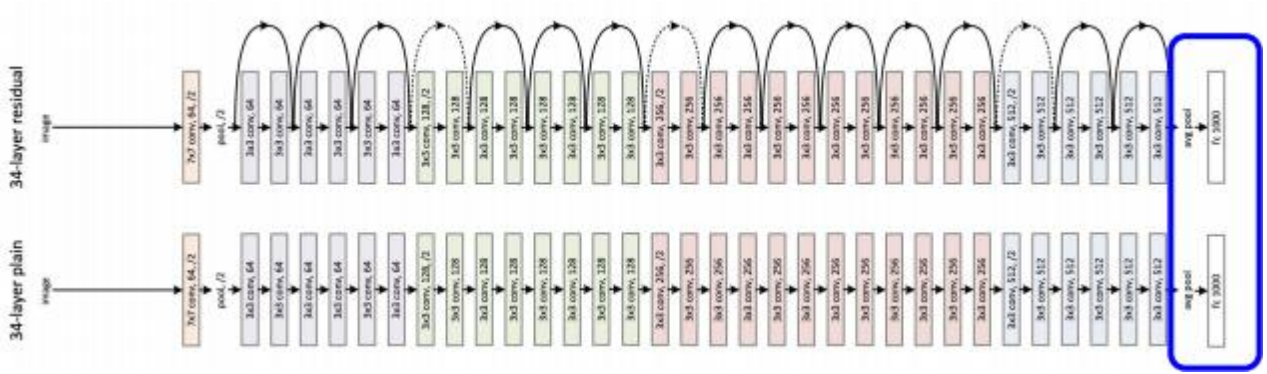


This image is CC0 1.0public domain



[Krizhevskyetal. 2012]

Linear layers



[He etal. 2015]

CIFAR10

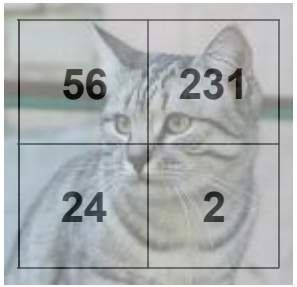


50,000 training images
each image is **32x32x3**

10,000 test images.

Example with an image with 4 pixels, and 3 classes (cat/dog/ship)

Flatten tensors into a vector



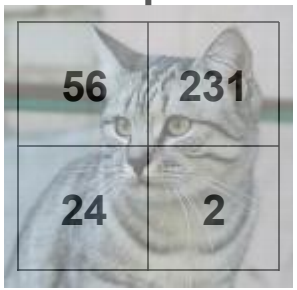
Input image

56
231
24
2

Example with an image with 4 pixels, and 3 classes (cat/dog/ship)

Algebraic Viewpoint

Flatten tensors into a vector



Input image

0.2	-0.5	0.1	2.0
1.5	1.3	2.1	0.0
0	0.25	0.2	-0.3

W

56
231
24
2

\+

1.1
3.2
-1.2

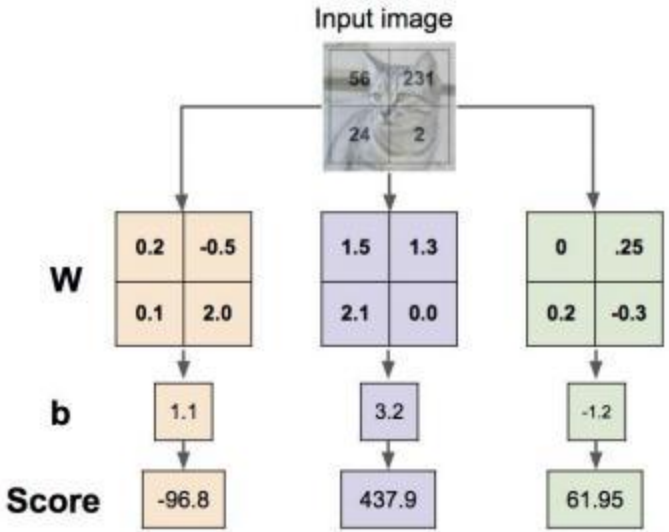
b

=

-96.8	Cat score
437.9	Dog score
61.95	Ship score

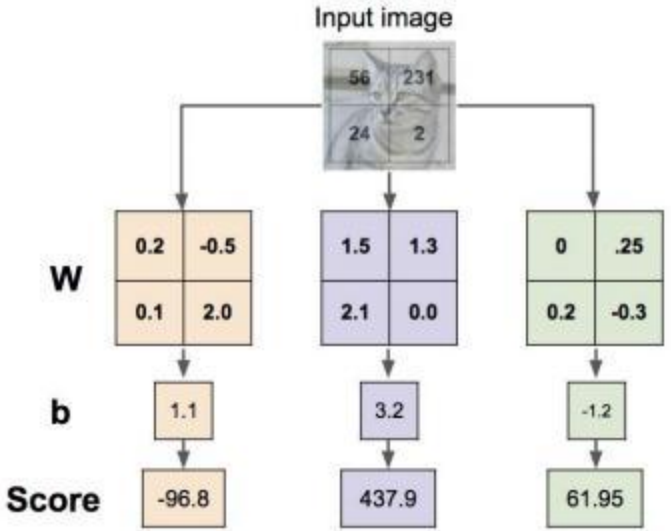
Interpreting a Linear Classifier





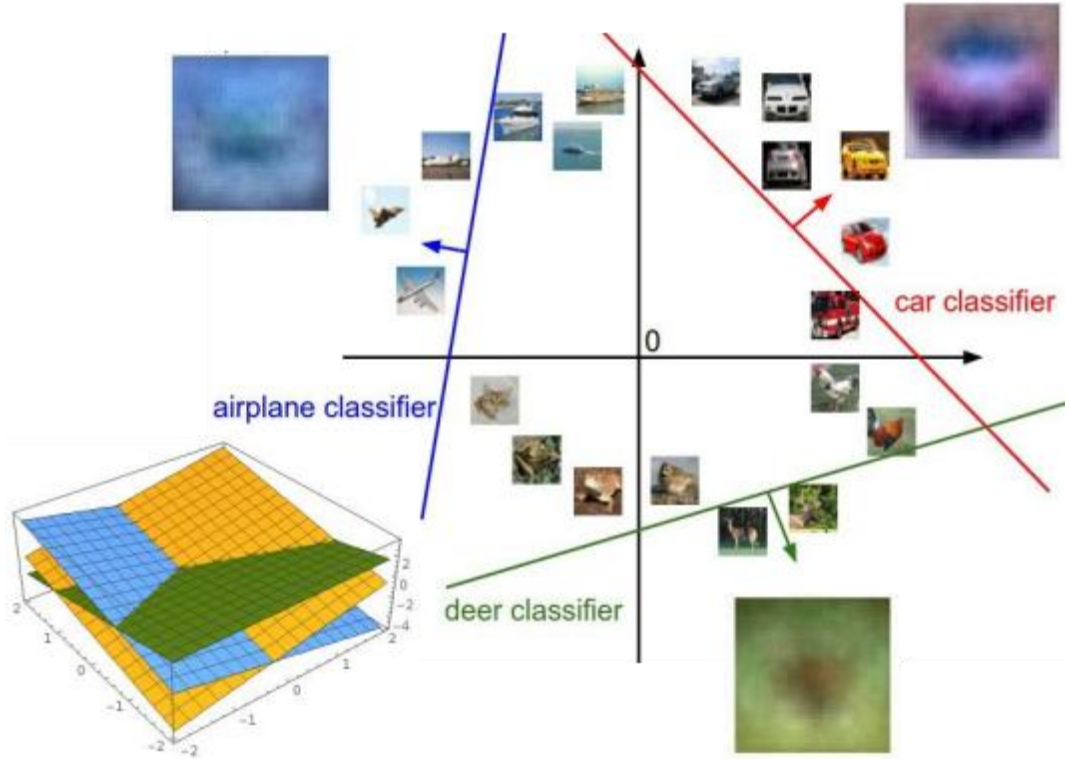
Interpreting a Linear Classifier: Visual Viewpoint







Interpreting a Linear Classifier: Geometric Viewpoint



$$f(x,W) = Wx + b$$



Array of **32x32x3** numbers
(3072 numbers total)

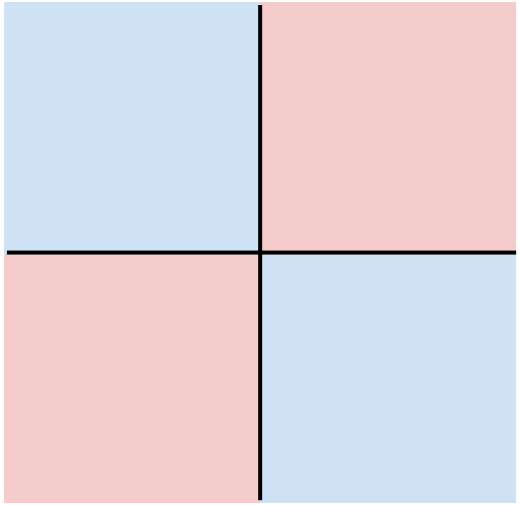
Plot created using Wolfram Cloud

Cat image by Nikita is licensed under CC-BY 2.0

Hard cases for a linear classifier

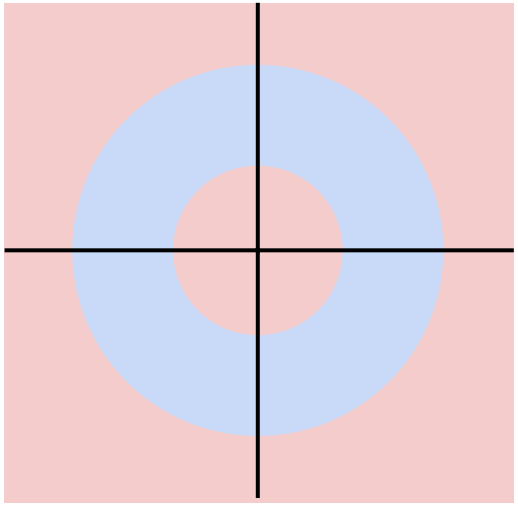
Class 1:
First and third quadrants

Class 2:
Second and fourth quadrants



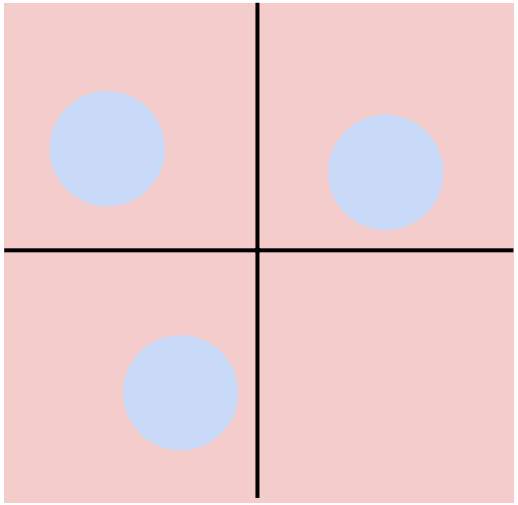
Class 1:
1 <= L2 norm <= 2

Class 2:
Everything else



Class 1:
Three modes

Class 2:
Everything else



Linear Classifier – Choose a good W





airplane	-3.45	-0.51	3.42
automobile	-8.87	**6.04**	4.64
bird	0.09	5.31	2.65
cat	**2.9**	-4.22	5.1
deer	4.48	-4.19	2.64
dog	8.02	3.58	5.55
frog	3.78	4.49	**-4.34**
horse	1.06	-4.37	-1.5
ship	-0.36	-2.09	-4.79
truck	-0.72	-2.93	6.14

Cat image by Nikita is licensed under CC-BY 2.0; Car image is CC0 1.0 public domain; Frog image is in the public domain

1. Define a **loss function** that quantifies our unhappiness with the scores across the training data.

2. Come up with a way of efficiently finding the parameters that minimize the loss function. **(optimization)**

Suppose: 3 training examples, 3 classes.
With some W the scores $f(x, W) = Wx$ are:





cat	**3.2**	1.3	2.2
car	5.1	**4.9**	2.5
frog	-1.7	2.0	**-3.1**

A **loss function** tells how good our current classifier is

Given a dataset of examples

$$\{(x_i, y_i)\}_{i=1}^{N}$$

Loss over the dataset is a average of loss over examples:

$$L = \frac{1}{N} \sum_i L_i(f(x_i, W), y_i)$$

Softmax classifier

Softmax Classifier (Multinomial Logistic Regression)



Want to interpret raw classifier scores as **probabilities**

cat	**3.2**
car	5.1
frog	-1.7

Softmax Classifier (Multinomial Logistic Regression)



Want to interpret raw classifier scores as **probabilities**

$$s = f(x_i; W)$$

$$P(Y = k|X = x_i) = \frac{e^{s_k}}{\sum_j e^{s_j}}$$

Softmax Function

cat **3.2**

car 5.1

frog -1.7

Softmax Classifier (Multinomial Logistic Regression)



Want to interpret raw classifier scores as **probabilities**

$$s = f(x_i; W)$$

$$P(Y = k|X = x_i) = \frac{e^{s_k}}{\sum_j e^{s_j}}$$

Softmax Function

Probabilities must be >= 0

cat	**3.2**	exp →	**24.5**
car	5.1		164.0
frog	-1.7		0.18

unnormalized probabilities

Softmax Classifier (Multinomial Logistic Regression)



Want to interpret raw classifier scores as **probabilities**

$$s = f(x_i; W)$$

$$P(Y = k|X = x_i) = \frac{e^{s_k}}{\sum_j e^{s_j}}$$

Softmax Function

Probabilities must be >= 0

Probabilities must sum to 1

cat	**3.2**	**24.5**	**0.13**
car	5.1	164.0	0.87
frog	-1.7	0.18	0.00

exp → normalize →

unnormalized probabilities

probabilities

Softmax Classifier (Multinomial Logistic Regression)

Want to interpret raw classifier scores as **probabilities**



$$s = f(x_i; W)$$

$$P(Y = k|X = x_i) = \frac{e^{s_k}}{\sum_j e^{s_j}}$$

Softmax Function

Probabilities must be >= 0

Probabilities must sum to 1

	Unnormalized log-probabilities / logits	exp	unnormalized probabilities	normalize	probabilities
cat	**3.2**		**24.5**		**0.13**
car	5.1		164.0		0.87
frog	-1.7		0.18		0.00

Softmax Classifier (Multinomial Logistic Regression)



Want to interpret raw classifier scores as **probabilities**

$$s = f(x_i; W)$$

$$P(Y = k|X = x_i) = \frac{e^{s_k}}{\sum_j e^{s_j}}$$

Softmax Function

Probabilities must be >= 0

Probabilities must sum to 1

L=-logp(y=|x=i)

	Unnormalized log-probabilities / logits	exp	unnormalized probabilities	normalize	probabilities
cat	**3.2**		**24.5**		**0.13**
car	5.1		164.0		0.87
frog	-1.7		0.18		0.00

L_i = -log(0.13)
= **2.04**

Softmax Classifier (Multinomial Logistic Regression)



Want to interpret raw classifier scores as **probabilities**

$$s = f(x_i; W)$$

$$P(Y = k|X = x_i) = \frac{e^{s_k}}{\sum_j e^{s_j}}$$

Softmax Function

Probabilities must be >= 0

Probabilities must sum to 1

L=-logp(y=|x=i)

	Unnormalized log-probabilities / logits	exp	unnormalized probabilities	normalize	probabilities
cat	**3.2**		**24.5**		**0.13**
car	5.1		164.0		0.87
frog	-1.7		0.18		0.00

— L_i = -log(0.13) = **2.04**

Maximum Likelihood Estimation
Choose weights to maximize the likelihood of the observed data

Softmax Classifier (Multinomial Logistic Regression)



Want to interpret raw classifier scores as **probabilities**

$$s = f(x_i; W)$$

$$P(Y = k|X = x_i) = \frac{e^{s_k}}{\sum_j e^{s_j}}$$

Softmax Function

L=-logp(y=|x=i)

	Unnormalized log-probabilities / logits	exp	Probabilities must be >= 0 unnormalized probabilities	normalize	Probabilities must sum to 1 probabilities	compare	Correct probs
cat	**3.2**		**24.5**		**0.13**		**1.00**
car	5.1		164.0		0.87		0.00
frog	-1.7		0.18		0.00		0.00

Softmax Classifier (Multinomial Logistic Regression)

Want to interpret raw classifier scores as **probabilities**



$$s = f(x_i; W)$$

$$P(Y = k|X = x_i) = \frac{e^{s_k}}{\sum_j e^{s_j}}$$

Softmax Function

L=-logp(y=|x=i)

	Unnormalized log-probabilities / logits	exp	Probabilities must be >= 0 unnormalized probabilities	normalize	Probabilities must sum to 1 probabilities	compare	Correct probs
cat	**3.2**		**24.5**		**0.13**		**1.00**
car	5.1		164.0		0.87		0.00
frog	-1.7		0.18		0.00		0.00

Kullback–Leibler divergence

$$D_{KL}(P||Q) = \sum_y P(y) \log \frac{P(y)}{Q(y)}$$

Softmax Classifier (Multinomial Logistic Regression)



Want to interpret raw classifier scores as **probabilities**

$$s = f(x_i; W)$$

$$P(Y = k|X = x_i) = \frac{e^{s_k}}{\sum_j e^{s_j}}$$

Softmax Function

	Unnormalized log-probabilities / logits	exp	Probabilities must be >= 0	normalize	Probabilities must sum to 1		Correct probs
cat	**3.2**		**24.5**		**0.13**	compare	**1.00**
car	5.1		164.0		0.87		0.00
frog	-1.7		0.18		0.00		0.00
			unnormalized probabilities		probabilities		

L=-logp(y=|x=i)

Cross Entropy

H(P,Q)=

$$H(p) + D_{KL}(P\|Q)$$